SYNEXUS SOFTWARE TECHNOLOGIES
Full Stack Development Track — Phase 1 Internship Evaluation

Week 2 Submission

Full-Stack Authentication & Protected Interfaces

Track	Duration	Format	Stack
Full Stack	Week 2 of 4	Practical Build	React + Express + MongoDB + JWT

Task Summary

This submission extends the Week 1 Inventory Management module with a complete authentication system. Users can register and log in through secure backend endpoints using JWT and password hashing; the session persists across page refreshes via an httpOnly cookie; and all inventory routes — both the API and the frontend pages — are now protected, redirecting unauthenticated users to the login screen.

Repository

github.com/Rahmatsoga/week1-task → Week2_Task/

1. What Was Implemented

Building on the Week 1 CRUD module, the following authentication requirements from the task brief were completed:

- Secure backend registration and login endpoints using JWT and bcrypt password hashing.
- Frontend login and registration screens with client-side validation and error handling.
- The auth token is stored in an httpOnly cookie and persists the session across a full page refresh.
- Frontend routes are protected: unauthenticated users are redirected to /login before any inventory data loads.
- Backend routes are protected: every /api/items endpoint now requires a valid session.
- Clear 401 Unauthorized handling for missing/invalid/expired tokens, distinct from 400/409 validation and conflict errors.
- Logout clears the session on both the client (shared auth state) and server (cookie invalidated).

Evaluation Criteria Coverage

Criterion	Marks	Status
JWT	5	Complete
Protected Routes	5	Complete
Session Persistence	5	Complete
Secure Token Storage	5	Complete
Authentication Flow	5	Complete

2. How It Was Implemented

2.1 New Files Added

```
server/
  models/User.js — user schema with automatic password hashing
  utils/generateToken.js — signs the JWT, sets it as an httpOnly cookie
  controllers/authController.js — register, login, logout, getMe
  middleware/authMiddleware.js — protect middleware verifying the JWT
  routes/authRoutes.js — /api/auth endpoints

client/src/
  context/AuthContext.jsx — global login state, shared across the app
  components/ProtectedRoute.jsx — redirects unauthenticated users
  pages/LoginPage.jsx, RegisterPage.jsx
  services/apiClient.js — shared Axios instance (withCredentials: true)
  services/authService.js — register/login/logout/getMe API calls
```

2.2 Password Security

Passwords are never stored in plain text. The `User` schema uses a Mongoose pre-save hook that automatically hashes the password with bcrypt (10 salt rounds) immediately before saving, and only re-hashes it if the password field was actually modified. Login verification uses a one-way comparison (`bcrypt.compare`) rather than ever decrypting the stored hash — the original password is never recoverable, even by the developer.

2.3 JWT-Based Sessions

On successful registration or login, the server signs a JSON Web Token containing the user's id, using a secret key (`JWT_SECRET`) that only the server holds. This token is set as an **httpOnly cookie** rather than being returned in the response body for the client to store manually. On every subsequent request, the browser automatically attaches this cookie, and the `protect` middleware verifies its signature and expiry before allowing the request to reach any inventory controller.

2.4 Why httpOnly Cookies Over localStorage

A common alternative is storing the JWT in `localStorage` and attaching it manually to each request. This was deliberately avoided: `localStorage` is readable by any JavaScript running on the page, so if the app were ever vulnerable to a cross-site scripting (XSS) attack, an attacker's script could read the token directly and impersonate the user. An httpOnly cookie cannot be read by JavaScript at all — only the browser and server can access it. The tradeoff is that this requires CORS to be explicitly configured with `credentials: true` on both the Express server and every Axios request (`withCredentials: true`), which this project has in place.

2.5 Protected Routes (Backend and Frontend)

On the backend, `router.use(protect)` is applied at the top of `itemRoutes.js`, so every inventory endpoint requires a valid session before its controller ever runs. On the frontend, `ProtectedRoute.jsx` wraps the Inventory page: while the app is still checking for an existing session it shows a loading spinner (avoiding a flash of protected content), and if no valid session is found, it redirects to `/login`, remembering the originally requested page so the user returns there after logging in.

2.6 Session Persistence Without localStorage

On every app load, `AuthContext.jsx` calls `GET /api/auth/me`. Because the httpOnly cookie is sent automatically by the browser, the server can identify the logged-in user without the client ever needing to store or manage a token itself. This is what allows a refreshed page to stay logged in.

2.7 Error Handling and Unauthorized Responses

Login and registration failures return distinct, correct status codes: `400` for missing fields, `409` for a duplicate email on registration, and a deliberately vague `401 "Invalid email or password"` on login failure — the message does not reveal whether the email exists, which prevents an attacker from using the login form to enumerate valid accounts. Any request to a protected route without a valid cookie receives `401 "Not authorized"` before reaching the controller. A pre-existing bug in the centralized error handler (status codes set via `res.status()` before a thrown error were being overwritten with a default 500) was identified and fixed as part of this week's work.

3. New Dependencies Installed

3.1 Backend

Library	Type	Purpose
bcryptjs	Production	One-way password hashing; passwords are never stored in plain text.
jsonwebtoken	Production	Signs and verifies the JWT used as the session token.
cookie-parser	Production	Parses the incoming Cookie header so req.cookies is available to read the auth t

3.2 Frontend

Library	Type	Purpose
react-router-dom	Production	Enables multiple pages (Login, Register, Inventory) with URL-based navigation a

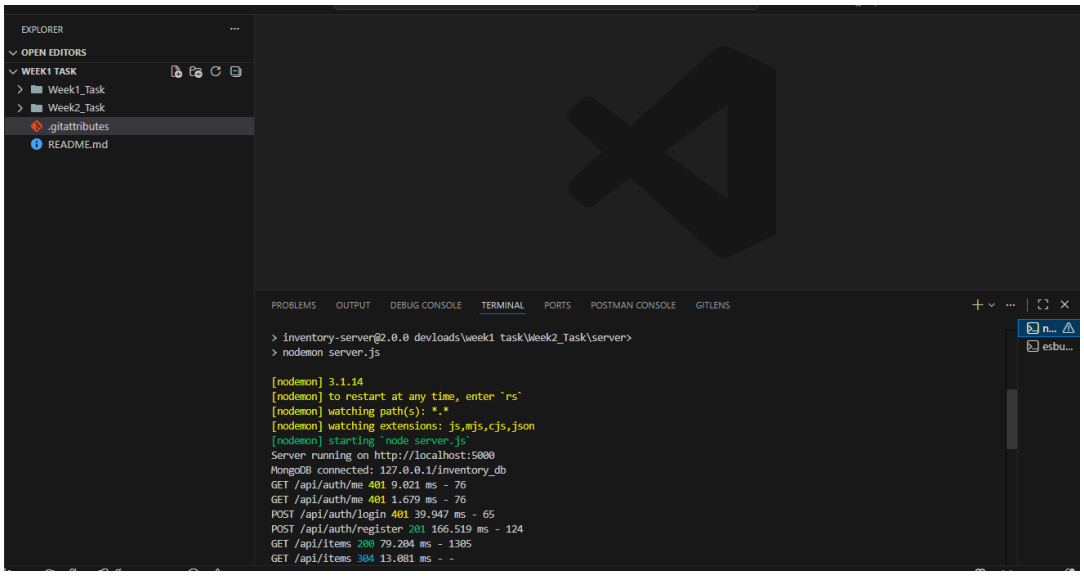
In addition, `vite`, `@vitejs/plugin-react`, `react`, and `react-dom` were pinned to specific stable versions (rather than "latest") to avoid a known native-binding crash affecting the newest Vite release on Windows, encountered during Week 1 setup.

4. Demo Walkthrough & Evidence

The steps below were followed to verify the reviewer checkpoint for Week 2: *"Reviewer can log in, refresh, access protected pages, and see unauthorized handling."*

4.1 Backend running with live authentication requests

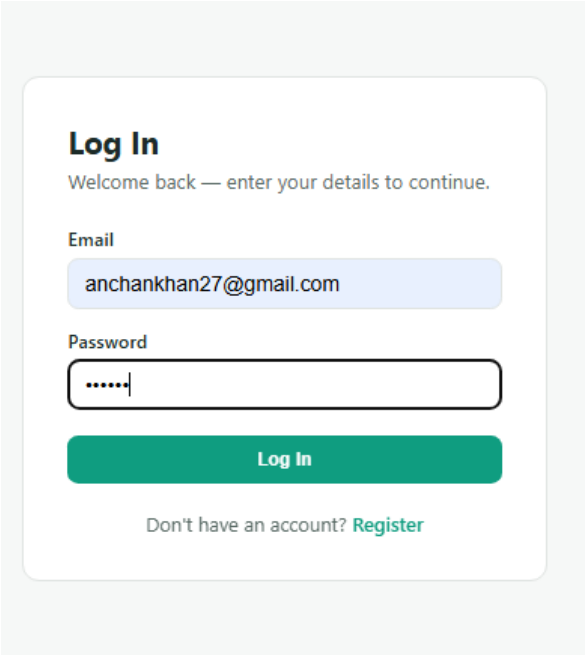
The server starts cleanly and connects to MongoDB. The request log below shows the real authentication flow in action: `GET /api/auth/me` returning `401` before login (no session yet), then a successful `POST /api/auth/register` returning `201`, followed by protected `GET /api/items` requests succeeding with `200` once authenticated:



Server running, MongoDB connected, and live auth request log (401 before login, 201 on register, 200 on protected routes).

4.2 Login page

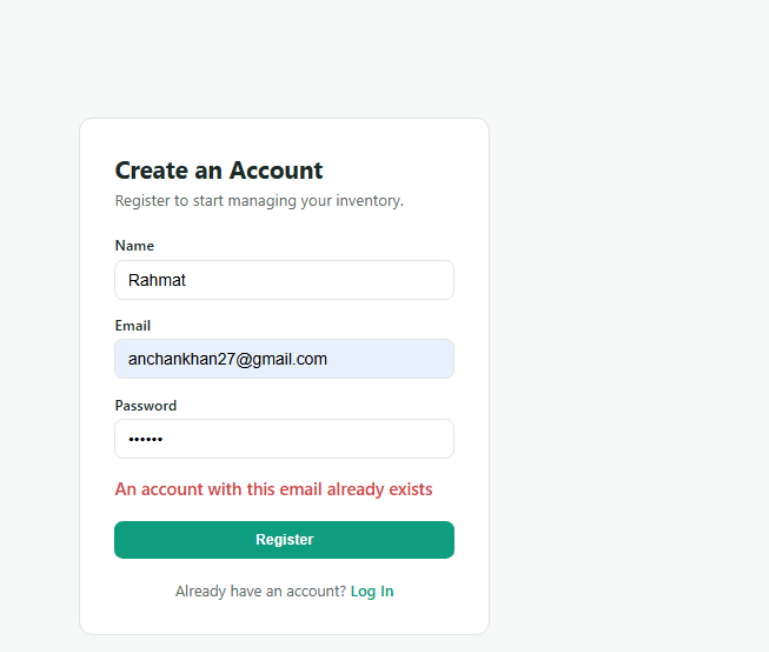
Unauthenticated visitors are redirected here automatically before any inventory data can be seen:



The Login screen, shown to unauthenticated visitors.

4.3 Duplicate email handling

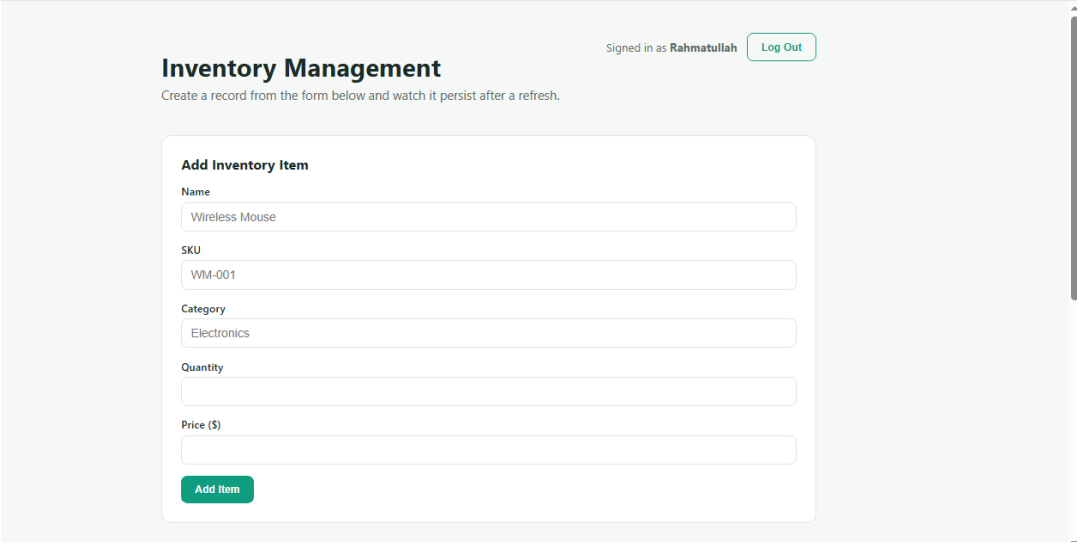
Attempting to register with an email that's already in use is rejected with a clear, specific error, confirming the 409 conflict response is correctly surfaced to the user:



Registration correctly rejected: "An account with this email already exists."

4.4 Authenticated Inventory page

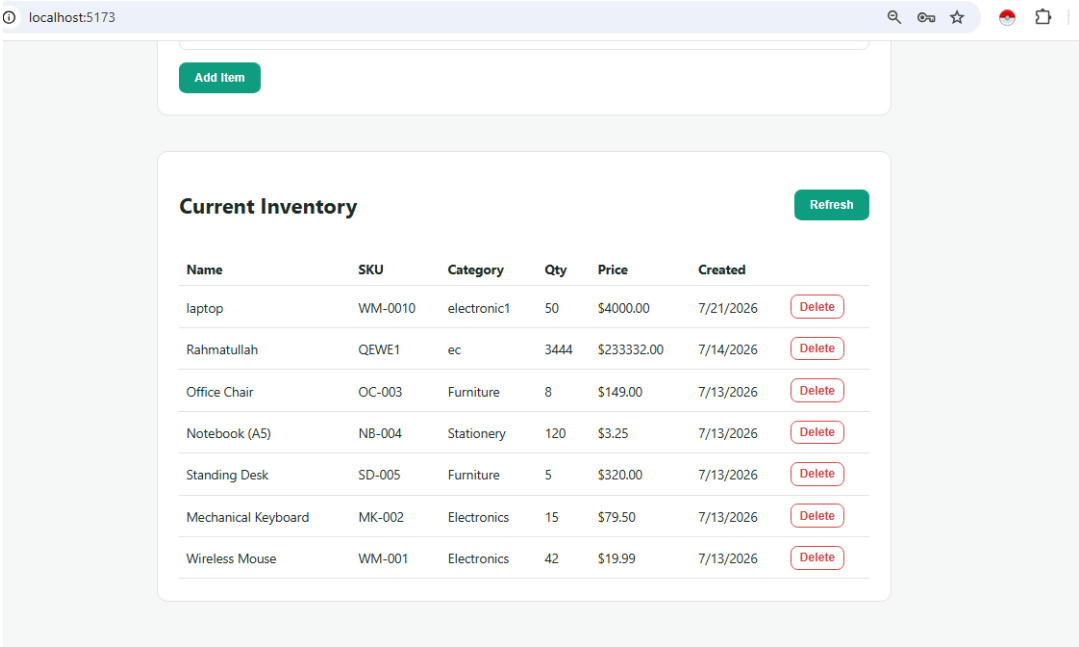
After a successful login, the protected Inventory page renders with a "Signed in as" indicator and a Log Out control — confirming the session and protected route are both working correctly:



Authenticated Inventory page, showing the logged-in user and Log Out button.

4.5 Inventory data loading for the authenticated user

The full inventory list loads correctly once authenticated, confirming the `protect` middleware allows valid sessions through to the underlying Week 1 CRUD functionality without any regression:



Inventory data loading correctly for the authenticated session.

4.6 Verified behavior

- Visiting the app while logged out redirects to /login rather than showing any inventory data.
- Registering a new account grants immediate access to the protected Inventory page.
- The "Signed in as [name]" indicator displays correctly.
- Refreshing the browser (F5) preserves the session — the user is not logged out.
- Clicking Log Out redirects to /login and the inventory page becomes inaccessible again.
- Visiting the protected route directly by URL while logged out redirects before any data loads.
- Registering with an already-used email returns a 409 conflict error, shown to the user.
- Logging in with an incorrect password returns a 401 error, without revealing whether the email itself exists.

5. Tradeoffs & Known Limitations

- No password reset / "forgot password" flow yet — out of scope for this week.
- No role-based access control; every logged-in user currently has identical permissions on inventory items.
- A single long-lived (7 day) JWT is used rather than a short-lived access token paired with a refresh token.
- No rate limiting yet on the login/register endpoints, which would help mitigate brute-force attempts.

6. Improvements With More Time

- Add rate limiting (e.g. express-rate-limit) on /auth/login and /auth/register.
- Add email verification on registration.
- Add short-lived access tokens with refresh token rotation for stronger security.
- Add role-based permissions if multi-user inventory sharing becomes a requirement.
